01 / IMPLEMENTATION PLAN

Jev × Helix

Complete research and implementation plan

A responsive decision system above a visually reactive controller. The goal is to preserve useful behavior while observations, human intent, and model advice change—not to add another model merely because a hierarchy is fashionable.

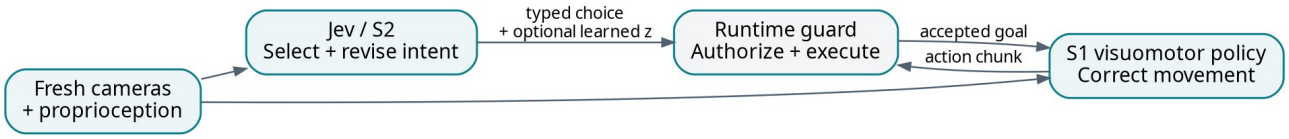


Figure 1. Proposed responsibility split. Jev selects intent; S1 adapts movement; the runtime controls commitment.

Where Jev earns its place S1 answers "How do I execute the current goal from this observation?" Jev answers "Is this still the right goal, what should happen next, and do I need more evidence?"

Situation	S1 contribution	Jev contribution
The target shifts slightly	Adjust the trajectory under the same goal.	Usually no new semantic decision.
"Use the blue box instead"	Execute the accepted replacement goal.	Interpret the revision; select a supported operation.
Old advice arrives after a correction	Must not independently revive old intent.	Assess relevant advice; runtime rejects obsolete revisions.
The task is ambiguous	Cannot resolve missing user intent by moving better.	Ask, wait, observe, or choose a justified alternative.

Research boundary: a capable native VLA may already solve some of these cases. Jev is valuable only if matched experiments show better task success, revision handling, calibration, or useful-response latency. A fixed pick-and-place alone is not sufficient evidence.

Design proposals unless labeled otherwise. No model training, controller deployment, or closed-loop experiment was executed while preparing this plan. Source references [R01–R15] are listed in Sections 25–26.

02 / IMPLEMENTATION PLAN

Requirements and reading map

The requirements are fixed; the implementation remains testable.

Foundation	Implementation obligation	Evidence required
Useful amid changing input and intent	Observe during execution; revise intent; preserve valid progress.	Changed-goal, disturbance, and ambiguous-history episodes.
Timing, uncertainty, revision, compute, commitment	Separate evidence from decisions and decisions from activated commands.	Timing traces, stale-result tests, explicit activation acknowledgments.
Selective use of evolving advice	Train useful/irrelevant/conflicting advice and reject stale authority in code.	Guidance stress tests with and without the advisor.

Required capabilities

Continuous observation; goal-directed selection; progress awareness; responsive interruption; calibrated abstention; bounded fallback on stale data or disconnect. ACT, OBSERVE, and THINK may occur together. ASK and WAIT are meaningful outcomes, not task failures by definition.

User-provided working constraints—not verified hardware facts

- RoboCasa checkpoint today; SO-101 target with an SO-100 reported available on campus. Real camera setup, access frequency, and available H100 credit remain unresolved.
- Mac M5 / 16 GB, two T4s, and an H100 were reported. These are resource-planning assumptions, not evidence that the complete stack meets its timing targets.
- No SO-101 demonstrations reported. An initial 50–100 demonstrations per skill is a collection hypothesis, not a universal sample requirement. Physical trials require an appropriate operating envelope and local stop path.

Read for…	Sections
Architecture and why Jev matters	3–9
Messages, state ownership, and concurrency	10–12
Data shapes and complete training program	13–16
Latency, worked events, and software	17–20
Deployment, evaluation, and delivery gates	21–24
Primary sources and evidence boundaries	25–26

Scope This is the full dependency-gated program, including learned latent conditioning, adaptive computation, deployment, evaluation, and later learning. Staged delivery is not a restriction to a basic first prototype.

03 / IMPLEMENTATION PLAN

Target architecture

Helix-inspired—not a reproduction of Figure's unreleased system.

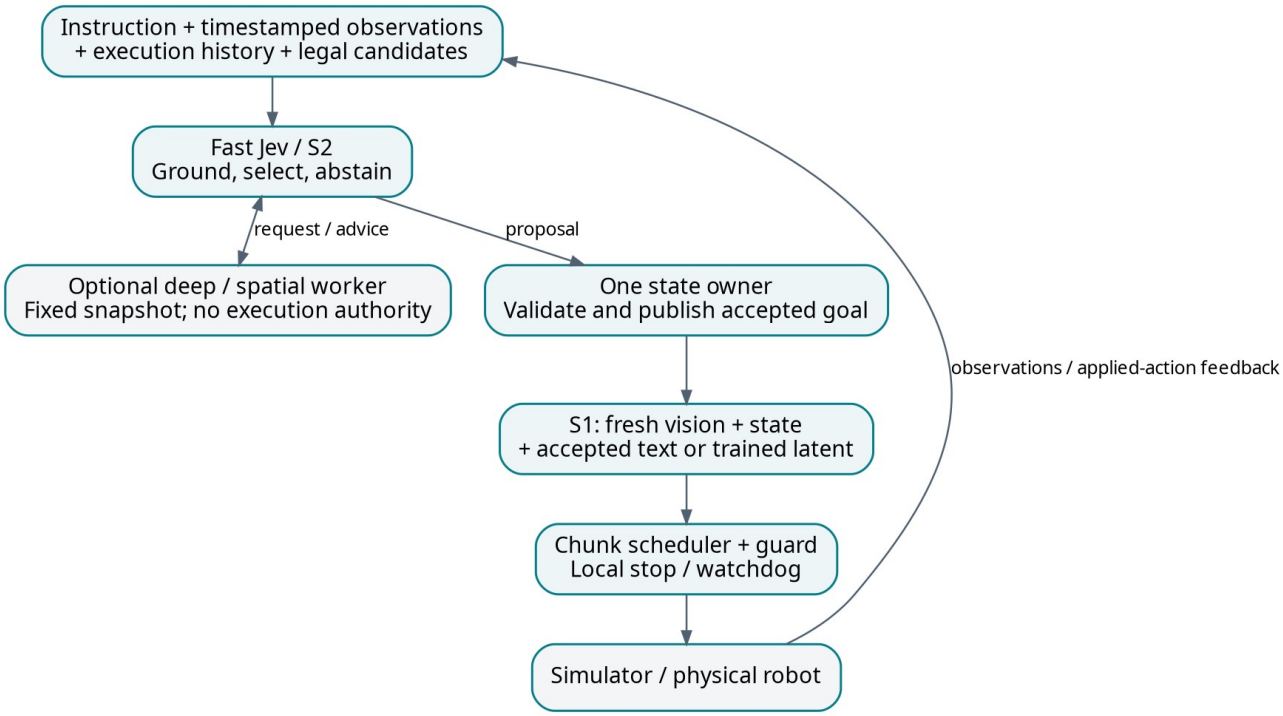


Figure 2. Proposed closed-loop architecture. The fast supervisor continues while optional analysis runs on its own snapshot.

Published anchor. Original Helix describes an S2 VLM and a faster S1 policy that both receive observations. S2 provides a learned latent; S1 uses newer local evidence. The pair is jointly trained with an observation-time offset. Figure reports 7–9 Hz for S2 and 200 Hz control for S1; those are not Fenon targets or measured camera-refresh rates. [R01]

Our extension. Jev adds explicit typed choices, evidence-aware abstention, optional computation, and advice interpretation. A single runtime owner controls desired intent. The executor owns the physically active command and applied-action index.

Do not conflate Helix's S1/S2 conditioning is not KAME-style advice selection. Adaptive depth is not part of the cited Helix design. Those are additional research hypotheses, each with its own training and ablation.

04 / IMPLEMENTATION PLAN

Responsibilities and authority

Separate semantic changes from local corrections.

Component	Owns	Must not claim
Jev / S2	Select a supported operation, assess completion, resolve references, ask/abstain, request extra computation.	Guaranteed physical feasibility or certified safety from a choice probability.
S1 policy	Observation-conditioned action generation and trained recovery/correction under an accepted goal.	Authority to change the user's goal or invent unsupported skills.
Optional advisor	Suggestions or evidence about a dated snapshot.	Permission to directly update the active command.
State owner	Instruction revisions, desired operations, result validation, event ordering.	That a sent command is already active, or that a transaction makes evidence true.
Executor + local guard	Action timing, chunk validity, activation acknowledgment, local limits and stop path.	That interpolating commands creates fresh visual feedback.

System behaviors

ACT: S1 executes an authorized skill. **OBSERVE:** sensing continues; a viewpoint change is itself a supported action. **THINK:** background computation runs when useful. **ASK:** Jev selects a question intent and arguments; a template or separate language renderer produces speech/text. **WAIT:** a validated waiting policy applies while sensing continues.

Example: identical pixels, different correct decisions

A cube is above box A. In history 1 the user still wants A, so continue. In history 2 the user has changed the destination to B, so revise. A motor policy without the revision cannot infer that history from the current pixels. A native VLA with complete intent/history might succeed; include that baseline rather than assuming Jev is necessary.

Candidate operations

```
CONTINUE_CURRENT
SWITCH_TO(skill_id, target_id, destination_id)
RETRY(skill_id, target_id)
ABORT_CURRENT / GUARDED_WAIT
ASK(template_id, object_ids)
```

Generate complete, compatible candidates in code. Requesting deeper computation is a separate control field, so THINK need not conflict with CONTINUE. "Retry" requires current-state support; it is not a magic recovery capability.

05 / IMPLEMENTATION PLAN

Two implementations, one hypothesis

Baseline integration and learned latent control are different experiments.

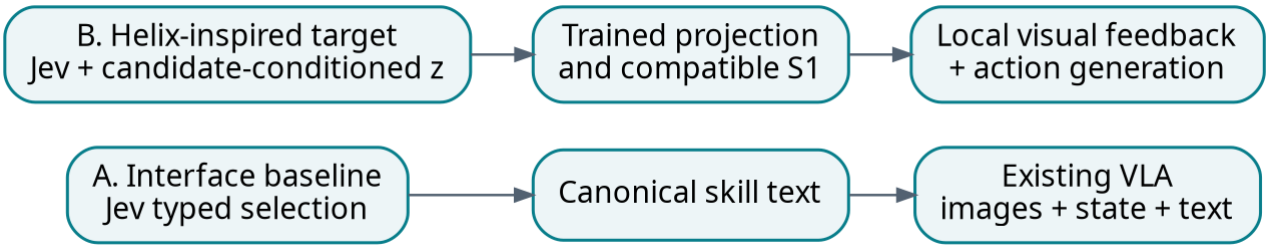


Figure 3. Compare the existing-instruction route against a trained latent route; do not confuse wrapping a VLA with replacing its S2.

	A. Existing-interface baseline	B. Helix-inspired research target
S2 output	Typed candidate mapped to canonical skill text.	Typed candidate plus selected-candidate-conditioned latent z.
S1	Existing image/state/text VLA, adapted to the embodiment.	Latent-conditioned visual policy; fresh local vision and state retained.
Training	Robot-specific execution and transition training as needed.	Conditioning projection, S1 adaptation, and joint or distilled S2/S1 training.
What it proves	Whether supervision and state protocol improve interaction.	Whether learned conditioning improves grounding, transitions, or cost beyond A.
What it does not prove	An external Jev removes the VLA's own VLM computation.	A learned latent is interpretable, safe, or interchangeable across checkpoints.

Choose an S1 route explicitly

Preserve an open VLA: add a residual latent-conditioning path to its action-conditioning interface and train it; retain native conditioning as the baseline. **Train a smaller visuomotor policy:** use a visual encoder, proprioceptive embedding, projected Jev latent, and action head. This is closer to the intended compute split but requires enough action data and a separately validated visual backbone.

Decision rule Keep A as the control throughout. Promote B only after it shows a material gain. GR00T/π/SmolVLA are possible starting points—not interchangeable executors and not evidence that Jev latents can be injected unchanged. [R07, R08, R11]

06 / IMPLEMENTATION PLAN

Jev / S2 model specification

One shared representation for spatial evidence and typed choice.

Proposed input: current instruction and revision; bounded event history; recent image/state window with masks and timestamps; active/desired operation; executor identity; legal candidates; compatible advice with source time. The model must receive only evidence available at the decision time.

```
H = backbone(images, state, instruction, history, candidates, advice)
p_choice = typed_readout(H, candidate_mask)
p_aux = spatial_and_progress_readouts(H)
c_selected = choose_or_abstain(p_choice, evidence_checks)
z = latent_readout(H, c_selected)  # only in learned-latent route
```

IMP to understand

Shared does not mean sequential. Spatial and decision readouts use the same internal evidence; the decision need not wait for an emitted spatial JSON. A post-selection latent readout may use one cheap extra projection, not another full backbone pass. Align its semantics with the selected candidate.

Candidate construction

Capability entries record the executor revision, canonical instruction, argument types, validated starting-state coverage, completion test, transition rules, and rollout results. A string seen in training is not a proven skill. Keep policy text likelihood separate from measured execution success.

- Enumerate only executor-supported skills and compatible arguments. Use dependable existing state to remove known-invalid options; uncertain evidence must not silently erase the only valid candidate.
- Include WAIT, ABORT, and ASK contracts. Validate object identity: metadata target IDs do not condition a text-only policy unless the instruction or trained interface binds them.
- Measure candidate recall independently. Joint selection cannot recover an omitted skill. Do not rebuild candidates from the same inference's decoded spatial outputs and still call the entire procedure single-pass.

WDYM above?

Readout limits

OneJev supplies typed model interfaces. MJev uses shared candidate scoring, but exact pointing/scalar refinement can require additional passes. A one-pass auxiliary head is a new trained approximation, not the original MJev procedure. [R03, R04]

Semantic bottleneck check A free continuous latent could cause S1 to ignore the selected skill. Condition z explicitly on that skill, test contradictory/shuffled/zero latents, and measure executed intent. Typed syntax alone cannot constrain every physical effect of a continuous latent.

07 / IMPLEMENTATION PLAN

Conditional spatial computation

Activate actual extra work—not a field named "reasoning."

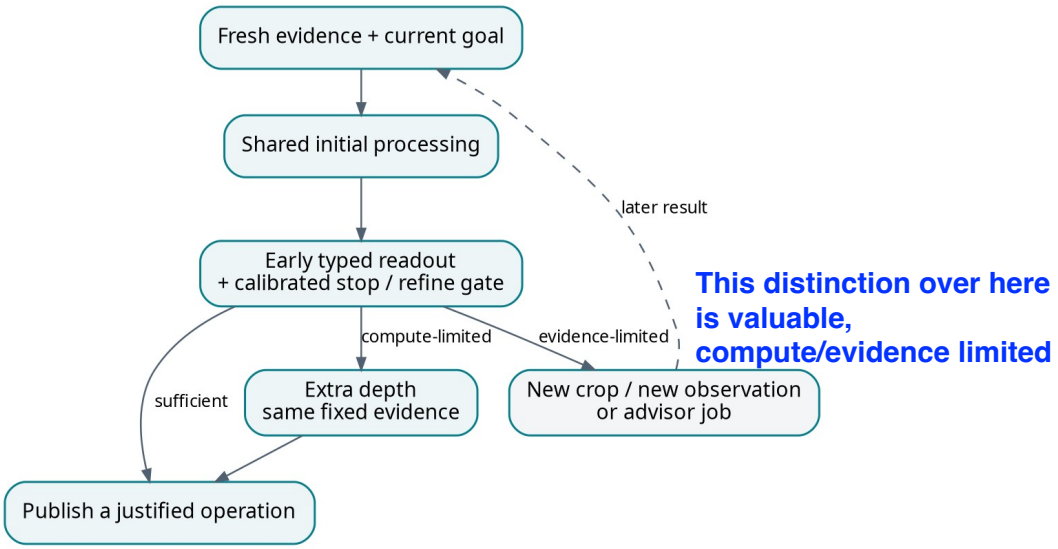


Figure 4. Distinguish insufficient computation from insufficient evidence. Extra loops do not reveal an occluded object.

Mechanism	Prerequisite	Key constraint
Early exit on a multimodal backbone	Train an intermediate readout and calibrate its exit policy.	A full expensive pass followed by a gate saves no backbone cost.
Adaptive depth LEV-style recurrent depth	Use a natively looped backbone, or train an explicitly recurrent adaptation.	Do not rerun dense OneJev layers arbitrarily and assume valid recurrence.
Targeted crop / spatial refinement	Train the visual-input/refinement contract and its stopping behavior.	More depth and higher visual resolution solve different limitations.
Separate deep worker	Independent request context and bounded scheduling.	Background work can still contend for the GPU; asynchrony is not isolation.

LEV reads choices at successive hidden-state loops. Its published stopper predicts agreement with the deepest answer, not physical success, and a deeper answer can be wrong. Its released study is not a ready-made multimodal robot supervisor. [R05]

Training and operational policy

Train all supported exits on meaningful choices. Use held-out outcome/oracle labels for deployment risk, with a fixed compute cap and an abstention path. Evaluate missed refinement as well as saved work. A provisional output may be "wait," not an unsafe motion just to appear responsive.

A loop deepening on snapshot k is not a newly updated belief at k+1. Keep the fast request path alive; terminate or discard stale refinements. Start with one early exit and one full route, then justify finer routing through measurements.

08 / IMPLEMENTATION PLAN

Spatial intelligence and observation

Do not build a geometry stack until the decisions require one.

Capability	Proposed owner	Data / constraint
Fresh motion correction	S1	Camera + proprioception aligned with action targets; disturbance examples.
Target semantics and relations	Jev	Image/history plus object-bound instructions and relation labels.
Holding and completion	Jev auxiliary heads or validated monitor	Temporal evidence; visibility/unknown labels; grasp/opening alone is not success.
Metric distance / clearance	Calibrated estimator when needed	Camera calibration, scale, synchronized views or valid depth sensing.
Persistent object identity	Tracker or explicit evidence association	Association confidence; handle occlusion and identical objects.
Executor feasibility	Empirical capability map / learned success model	Actual rollouts from current and predecessor end states; not scene facts alone.

MJev is a learned visual decision model, not a switchable stack of detector, depth, and reconstruction services. Its choice, numerical, and pointing contracts differ; an emitted scalar is not a calibrated physical measurement. [R04]

Camera changes have consequences

In simulation we could make of the simulators report itself, not sure on rea hardware though

- Overhead + wrist cameras are multiple views, not automatically a calibrated fixed-baseline stereo pair. A moving wrist requires time-varying camera-to-robot geometry for explicit triangulation.
- Figure's logistics report describes stereo feature fusion before tokenization and multiscale S1 features. Adopting this requires the appropriate camera layout, data, and retraining; it does not remove visual-encoder cost. [R02]
- If sim depth is used for teacher labels, keep it out of deployment inputs unless real depth is available. Record occlusion, calibration revisions, units, masks, and source frame IDs.

Three required judgments

Held object: identity distribution plus unknown. **Location:** 2D point/box or relation with declared view/frame; metric position only when supported. **Completion:** evidence over a window with explicit success conditions and separate uncertainty. Evaluate their errors independently of the downstream decision.

where it could break
- Timing mismatch. If the joint reading is 30 ms older than the image and the arm is moving, the computed camera pose is wrong, so the 3D point is wrong.

- Calibration error. A few millimetres or a degree off in the hand-eye offset becomes centimetres of error at the object.

- Joint encoder error. Cheap arms like the SO-100/101 have some slop, which also propagates into the camera pose.

09 / IMPLEMENTATION PLAN

S1 and the learned interface

What stays, what changes, and what must be trained.

S1 remains a closed-loop action model. It receives current local observations, not only S2's old semantic vector. It must learn the accepted instruction format, embodiment, action timing, and transitions. A motor servo beneath S1 has a separate rate from action-policy inference and camera capture.

```
z_k = E_jev(observation_at_k, instruction, selected_candidate)
u_t = P(z_k, age_of_z, masks, mode)
A_t = action_policy(local_images_t, local_state_t, u_t, prefix)
```

The proposed projection P and age/mode inputs must be trained. Helix's published latent communication and end-to-end action loss motivate this route, but do not supply an open compatible encoder/decoder pair for this project. [R01]

S1 modification	Required follow-through
Keep π / SmolVLA text conditioning	Map selected skill to supported text; measure direct native VLA baseline; adapt camera/state/action transforms.
Add z to a pretrained action head	Audit exact conditioning interface, masks and normalization; train projection and trainable receiving layers.
Remove the original VLM / text backbone	Replacement training or distillation of its conditioning role; this is not a runtime optimization.
Use a small direct visuomotor policy	Pretrain/adapt visual features; train task-conditioned actions; independently establish skill coverage.
Change action head or representation	Regenerate targets, normalization, decoder and controller mapping; retrain and rerun closed-loop tests.
Add autonomous ASK/THINK outputs to S1	New typed decision training and authority rules. Not needed when Jev owns supervisory choices.

- identtify the layers to train for latent instructions
- Remove VLM backbone??
- Visuomotor policy: pi is already trained on some subtasks
- Changes to action head?

Output semantics

Action dimensions, units, absolute/delta convention, reference frame, gripper scale, control interval, and padding mask are checkpoint-specific. Never silently truncate a Panda action vector to fit an SO arm. Keep model output decoding and physical command validation distinct. [R07, R09]

Message contracts

Proposed protocol, not an existing vendor API.

All messages have a schema revision. Source timestamps describe when evidence was observed, not when an expensive job finished. A session epoch changes after restart so pre-restart commands cannot reappear.

```
ObservationSnapshot:
  episode_id, session_epoch, observation_id
  camera_frames: [{camera_id, buffer_ref, capture_ns, valid}]
  state: {q, dq_optional, gripper, sample_ns, units, valid_mask}
  instruction_rev, instruction_text
  active_command_id, desired_command_id, applied_action_index
  progress_summary, available_candidates, compatible_advice
DecisionProposal:
  request_id, episode_id, session_epoch, instruction_rev
  source_observation_id, source_capture_ns, base_active_command_id
  candidates_rev, selected_candidate_id, candidate_scores
  evidence_status, requested_compute, latent_ref_optional
  latent_encoder_rev, latent_interface_rev, executor_rev
  expires_ns
AcceptedGoal:
  command_id, instruction_rev, selected_candidate_id
  skill_text, target_ids, mode
  conditioning: {text | latent_ref, interface_rev, source_ns}
  transition_policy, expires_ns

ActionChunk:
  command_id, executor_rev, conditioning_id, source_observation_id
  start_action_index, dt_ns, actions[H, Da], valid_mask

ExecutionAck:
  command_id, status, activation_ns, last_applied_action_index
```

Clock and buffer contract

Use an agreed monotonic timeline for durations. Across machines, map source clocks with measured offset/uncertainty; do not subtract unrelated monotonic timestamps. Keep wall-clock times for audit. Frame and latent references must remain valid until their consumers finish.

Advice is evidence—not execution authority

Advice carries its original instruction revision, operation context, source frames, scope, and expiry. It can contain text or structured suggestions, but cannot introduce an executable skill outside the capability contract. Jev decides how to use compatible advice; the owner still validates its resulting proposal.

Latent compatibility Matching shape is insufficient. Load encoder, projection, policy, tokenizer/preprocessor, normalization, and interface revision as a compatible bundle. A latent must not silently substitute for an instruction string or an arbitrary KV cache.

State ownership and concurrency

Small in-memory records; no multi-agent writable knowledge database.

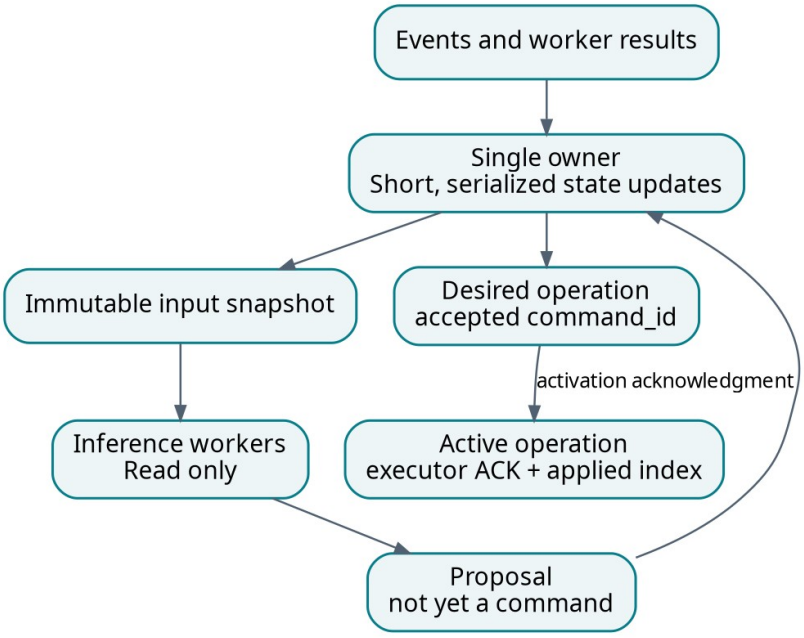


Figure 5. Workers compute on snapshots. One owner publishes intent; executor acknowledgment establishes active behavior.

Record	Meaning / update authority
Instruction revision	Current accepted user intent. Owner increments on meaningful instruction changes, not camera frames.
Desired operation	Accepted next command. Only the owner can change it.
Active operation	Executor-reported activated command; never inferred merely from dispatch.
Evidence and progress	Dated estimates, confirmed events, unknown states. Model certainty is not sensor truth.
Latest observations	Bounded read-only buffer references with independent sensor timestamps.
Outstanding jobs	Request IDs, base context, cancellation/expiry; workers return results only.

Shared features within one forward pass are not a database. Persistent inference caches need request isolation and explicit ownership. Persistent task records need ordered update rules. None of these makes a model's interpretation correct.

Concurrency invariant The owner must never block on GPU inference. Use normal queues first. Shared tensors require correct lifetime and publication semantics; GPU consumers must wait on readiness events. "Single writer" does not mean "no synchronization."

State transitions and stale-result rules

Causal validity beats "the last job to finish wins."

Event	Owner behavior	Executor behavior
New user instruction	Advance instruction_rev; invalidate incompatible pending intent; trigger fast decision.	Enter transition policy or guarded behavior; do not blindly finish old intent.
New camera frame	Update snapshot reference and timestamps; do not change instruction_rev.	Use fresh input on the next eligible policy update.
Deep result arrives	Check session, instruction, operation scope, evidence age and relevant changes; otherwise discard.	No direct action from advice.
Accepted replacement	Create command_id; desired changes. Active remains old until ACK.	Replace only valid future commands under the transition policy.
Completion estimate	Mark as evidence until completion criteria are satisfied.	Report applied actions and observed executor status.
Disconnect / expiry / stop	Mark unavailable; cancel incompatible work and log reason.	Local watchdog/stop behavior takes precedence, with finite stopping time.

Revision rules that prevent subtle bugs

- Track instruction revision, operation/command ID, and conditioning revision separately. A deeper latent refresh for the same operation is not necessarily a new user instruction.
- Do not blindly reject every result based on a changed active command: require its scope to match the operation it judges. Conversely, a same-instruction result may still be stale after a grasp/drop event.
- Assign unique command IDs; make acknowledgments and retries idempotent. Reject action chunks from canceled commands and elapsed action intervals.
- Do not update latents in-place while S1 reads them. Publish a completed immutable conditioning bundle; retain it while any consumer references it.
- Never promote a provisional prediction to "completed" because another model echoed it. Confirm through independently specified evidence and track contradictory observations.

History and recovery

Maintain a bounded current record plus an append-only audit log for replay, not a general graph database. After restart, begin a new session epoch, reconcile actual robot state, and require a valid new command before resuming. Store sensitive camera logs under explicit retention/access rules.

Training data: concrete shapes

Canonical research schema; adapters map it to each model's required format.

Field	Canonical shape / type	Meaning
RGB frames	[T, V, Himg, Wimg, 3], uint8	Time-major RGB; per-frame capture times and view masks.
Proprioception	[T, Dq], float32	Named measured channels. Derived velocities need their own validity and method.
Expert commands	[T, Da], float32	Commands in declared frame, units, control mode and sample interval.
Candidate set	Ragged K strings/IDs + valid mask	Recorded before the decision; include legal non-motion alternatives.
Decision supervision	[Ndecision, K] valid-choice mask / labels	Allow multiple acceptable actions where justified; label abstention.
Spatial supervision	object labels; [N,2] points; [N,4] boxes	Declared camera/frame; observation visibility and annotation masks.
Events / guidance	Timestamped JSONL records	Instruction revisions, advice delivery, ACKs, applied indices, success/failure.

Worked tensor example—not checkpoint defaults

```
B=8, V=2 views, Tobs=2, image=224x224
S2 image batch:       [8, 2, 2, 3, 224, 224]  # B,Tobs,V,C,H,W
S1 latest-frame view: [8, 2, 3, 224, 224]
State example:       [8, 6]  # only after mapping 5 joints + gripper
Candidate IDs:       [8, K] plus masks; tokenize descriptions separately
Proposed latent:     [8, 1, 256]  # Nz=1, Dz=256 are hyperparameters
Action target:       [8, 16, 6]   # H=16 is experimental, not stock H
```

Do not feed Tobs=2 into a single-frame model without an adapter/trained temporal interface. Real dimensions and gripper convention must come from the calibrated robot. Missing channels are masked, not invented.

For contrast, the supplied RoboCasa model card lists three 256×256 camera inputs, a 16-dimensional state, and a 12-dimensional action. That is not the SO-arm contract. [R09]

Causal alignment

Record commands sent and actually applied separately. Join frames/state to decision time without future leakage; preserve timestamp uncertainty. Build train/validation/test splits by episode, scene, and operator before extracting overlapping windows. Keep simulator oracle signals in labels/evaluation only unless available in deployment.

Data collection and temporal examples

Static successful demonstrations do not cover revised intent.

Data family	What to collect	Required labels
Atomic execution	Supported skills across objects, viewpoints, starts and speeds.	Expert actions, task boundaries, success conditions.
Skill handoffs	Successor starts from actual predecessor end states, not only reset poses.	Previous/next skill, transition interval, outcome.
Instruction revisions	User changes target or cancels during different execution phases.	Event time, revised valid candidates, actual corrected actions.
Disturbances / recovery	Slips, displacement, occlusion, failed grasps and blocked routes.	Failure onset, observed evidence, correct recovery or abstention.
Delayed / conflicting advice	Useful, irrelevant, old-goal and same-goal-wrong guidance.	Delivery/source times, relevance, valid decision after arrival.
Compute decisions	Cheap/full outputs with independent outcome labels.	Whether extra depth or new evidence actually helped.

One event-training example

```
t=0.00: goal=A; cube held; expert carries toward A
t=0.32: user changes destination to B; instruction_rev increments
t=0.34: fast target = GUARDED_WAIT or supported SWITCH_TO(B)
t=0.45: old advice recommends A; target = IGNORE_SUPERSEDED
t=0.52: executor activates B; expert transitions from actual state
t=1.10: cube slips; target = ABORT_CARRY + supported reobserve/recovery
```

Times illustrate an event trace, not measured latency. The state-dependent targets must be generated by a competent controller/teleoperator or reviewed oracle. A switch may require a pause; do not prescribe it as universally safe.

Rules for useful training data

- Do not relabel actions toward A as actions toward B. Collect the correct continuation from the intermediate state or simulate it with a validated teacher.
- Use random irrelevant advice as augmentation, plus hard same-goal contradictions. Keep unrelated sampled guidance inside the training split. KAME randomized guidance supplies the speech-domain precedent, not robot labels. [R06]
- Record the causal event before generating teacher outputs. Hindsight text may label a skill, but must not leak future state into the model's online input.
- Retain corrective demonstrations and validated safe failures; do not train action imitation to reproduce uncontrolled failure trajectories as successes.

S2 training program

Joint grounding and typed supervision, followed by adaptive computation.

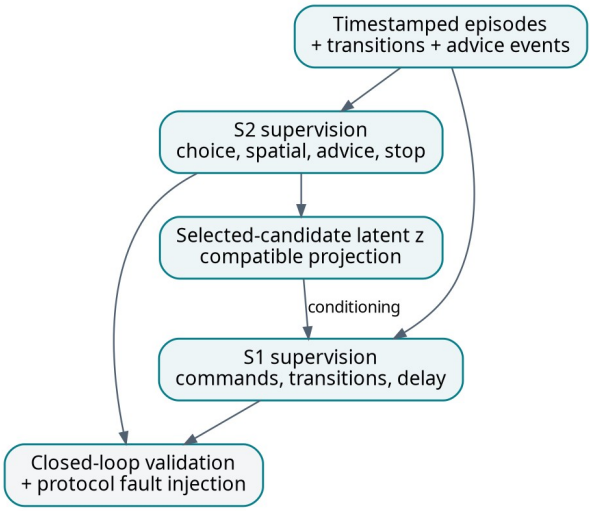


Figure 6. Forward conditioning links the models. During joint training, action gradients flow backward from S1 through z into trainable S2 parameters.

Stage	Train / freeze	Exit condition
S2-A: direct decision baseline	Evaluate a pinned multimodal Jev; freeze initially.	Known performance on oracle-state and visual-state cases.
S2-B: grounded supervision	Train decision/auxiliary readouts and chosen language-side adapters; retain visual weights initially.	Better held-out choice/grounding without unacceptable calibration loss.
S2-C: guidance and history	Train timestamped guidance use, unknowns, completion and revision examples.	Reject irrelevant advice; retain useful advice under delay.
S2-D: adaptive depth	Train all exits/loops and a calibrated stop/refine policy.	Quality–latency frontier improves over matched fixed-depth alternatives.
S2-E: latent alignment	Joint or distilled action conditioning, while retaining typed losses.	Executed intent matches the authorized candidate.

```
L_S2 = L_choice + w_cal * L_Brier + w_aux * L_spatial
       + w_prog * L_progress + w_adv * L_advice + w_exit * L_exit
```

Proposed loss family; coefficients are tuned on validation, not supplied by a source. Mask unavailable labels. Use classification loss for discrete judgments, appropriate coordinate/scalar losses for continuous labels, and separate unknown/visibility supervision. For multiple valid choices, optimize probability mass over the acceptable set rather than enforcing an arbitrary unique answer.

Train the gate against useful decisions or validated full-path gains, not confidence alone. Calibrate separately after quantization and architecture changes. Account for candidate order/size effects. OneJev probability is not executor success probability; learn success estimates from policy rollouts if needed. [R03]

S1 and latent-interface training

Learning the physical meaning of Jev's conditioning.

Curriculum and gradients

Stage	Implementation	What must remain controlled
S1-A: embodiment competence	Adapt the existing policy using native text/image/state conditioning.	Action decoder, dt, camera mapping, normalization, task definitions.
S1-B: temporal robustness	Train real handoffs, goal revisions, observation offsets and committed prefixes.	Correct expert continuation; old actions are context, not new-goal targets.
S1-C: latent receiver	Add a trained projection/cross-attention path; train receiver layers before broader unfreezing.	Checkpoint/interface version; source latent available at decision time.
S1-D: joint alignment	Backpropagate action loss through z into permitted S2 parameters; retain typed/spatial supervision.	No silent semantic drift; compare against text-only baseline.
S1-E: deployment adaptation	Fine-tune/validate realistic jitter, missing observations and approved precision.	Closed-loop outcome quality, not only action-vector similarity.

Choose the appropriate action objective

Keep the native flow/diffusion objective when adapting that class of VLA. For a newly trained deterministic action regressor, use dimension-aware action regression and supervised completion; apply trajectory or smoothness penalties only where they preserve valid behavior. A pretrained flow head is not interchangeable with a direct regression head. [R07, R08]

```
At time t, sample an earlier S2 context t-delta.
Compute z from that context and the accepted candidate.
S1 sees current images/state + z + latent age + committed prefix.
Supervise the valid uncommitted continuation from the actual state.
```

A latent from a superseded instruction is not ordinary timing jitter. Train controlled transition handling, and enforce revision validity in the runtime. If per-loop latents can reach S1, train S1 on every supported exit depth or distill them to a stable interface.

Failure tests for latent conditioning

- Shuffle, zero, contradict, and age z. Correct goal-conditioned behavior should depend on valid conditioning—not ignore it or obey contradictory latents.
- Train with a mix of teacher and on-policy selected conditions to reduce exposure mismatch; use valid action labels. Freezing the entire wrong receiver cannot teach a new latent meaning.
- Store cached training latents with encoder/adapter revision. Joint updates invalidate those caches unless deliberately using a frozen teacher.

Timing, action chunks, and interruption

Continuous motion is not proof of response to new evidence.

```
L_event→command = capture + queue_wait + decision
                  + executor_update + delivery + handover
```

This is a path decomposition. Measure the actual critical path; overlapping work is not added twice, and the sum of component p95 values is not the system p95. Same-goal corrections can bypass Jev; changed intent usually cannot.

Deadline from the discussion	Interpretation in this plan
Acknowledgment ≤200 ms	User event to acknowledgment. Do not count this as completed replanning.
Behavior change p50≤70 ms; p95≤100 ms	Target for validated foreground response. Report S1-only disturbances separately from intent-change paths.
New subtask selection ≤1 s	Planning objective; guarded response may occur earlier. Incompatible with claiming every new semantic action begins within 100 ms.
Safety stop	Independent local mechanism with measured finite stopping time/distance; not a probabilistic latency target.

Committed-prefix example

```
Example only: action interval = 20 ms; observed compute delay = 60 ms.
Time:       0       20       40       60       80 ms
Old chunk: [A0]     [A1]     [A2]     [A3]     [A4]
New job:   [----------- computing -----------]
Used:      [A0]     [A1]     [A2]     [B3]     [B4]
```

Three old actions elapse during computation. Condition the new trajectory on the prefix; replace future actions only under the configured transition policy. RTC inpainting and training-time prefix conditioning address this compatibility problem. They do not guarantee safe arbitrary goal switches. [R10]

Operational controls

- Log inference start/end, frame capture, advice source/arrival, dispatch, ACK, and first applied new-goal command. Record p50/p95/p99, deadline misses, and evidence age.
- Use real measured delay distributions for training, including jitter and contention. A growing action buffer can hide reaction delay behind smooth motion.
- Never average trajectories from incompatible goals as a default repair. Stop/guard logic can override chunk commitment; physical motion still takes time to arrest.

Worked example: revised destination

S1 stays visually reactive; Jev owns the semantic change.

Event	Jev / owner	S1 / guard
1. User: “Put the cube in A.”	Select supported pick/place sequence; command 41 accepted.	Uses fresh images to grasp and carry.
2. Box A shifts slightly.	No new intent required if current goal remains valid.	Tracks the new placement geometry within its learned capability.
3. User: “Actually, B.”	Increment instruction revision. Invalidate incompatible desired work immediately.	Apply validated pause/transition policy; do not silently finish in A.
4. Jev chooses place-in-B.	Verify target binding and capability; publish command 42.	Generate compatible continuation from actual state; ACK activation.
5. Old advisor recommends A.	Reject instruction mismatch. Active command remains 42.	No old latent or chunk may reactivate A.
6. Cube released into B.	Update progress only after completion evidence; select next step if any.	Report execution and retain sensor feedback.

Why each component is necessary

Jev’s useful work is steps 3–5: it integrates changed intent and advice with progress. S1’s useful work is the local motion in steps 1–2 and the trained transition in step 4. The owner prevents old computational results from regaining authority. These capabilities can be compared against a native VLA receiving the same instruction/history; do not assume that baseline cannot do them.

Training required

- S2: destination-change examples, target identity, stale advice rejection, and alternate supported choices.
- S1: new-goal transitions from carrying configurations, including clearance constraints and guarded pauses—not just clean reset-to-place demonstrations.
- Runtime: deterministic stale-result, command-ID, cancel/ACK, and delayed-chunk tests.

Incomplete information If B is ambiguous or unreachable under the current capability contract, Jev chooses ASK/WAIT rather than fabricating geometry. The question renderer may use “Which blue box?” while S1 follows the validated guarded behavior.

Worked examples: failure and uncertainty

Correctly refusing to guess is part of useful interaction.

Scenario	Correct division of work	Required evidence / training
Object slips during carry	S1 may make a trained local correction; Jev aborts invalid carry intent and selects supported recovery.	Slip visibility, gripper/object history, actual post-slip recovery actions.
Object disappears behind hand	Jev distinguishes occluded from absent; request more evidence or wait.	Unknown labels, temporal persistence, viewpoint-change capability if used.
Advisor says “step complete” but object remains held	Jev rejects advice contradicted by current evidence; owner does not mark completion.	Same-goal misleading advice and independently defined success labels.
Geometry is clear, instruction is ambiguous	Ask the user rather than repeatedly running spatial refinement.	Ambiguous reference pairs and meaningful clarification templates.
Compute or network disconnect	Owner expires incompatible work; local executor enters context-appropriate fallback.	Watchdog tests; holding a fragile object is not equivalent to an empty gripper.

Example: what the supervisor sees

```
holding: {candidate: cube_7, probability: 0.81, visible: partial}
step: {id: carry_7_to_A, status: running}
completion: {estimate: uncertain, source_frames: [1480, 1482]}
advice: {suggestion: done, source_frame: 1461, instruction_rev: 8}
current_instruction_rev: 8
```

Illustrative values. Same instruction revision does not make the old completion advice true. The correct action may be another observation, continuation, or a guarded pause depending on the actual physical situation.

Why more compute is not always the answer

A deeper loop can improve reasoning about available evidence. It cannot recover a missing camera view or the user’s unspoken intent. The compute controller therefore chooses between more processing, new evidence, clarification, and a current justified action. Train and evaluate those alternatives rather than equating low confidence with a larger model call.

Software and execution structure

Proposed repository boundaries; not claims about the existing Fenon code.

```
jev_control/
  contracts/       # schemas, versions, calibration / action conventions
  perception/      # acquisition, timestamp alignment, optional trackers
  supervisor/      # Jev readouts, candidates, adaptive-depth policy
  advice/          # asynchronous jobs; no command authority
  runtime/         # state owner, validator, dispatch, audit log
  executor/        # VLA adapters, chunks, local guard, acknowledgments
  training/        # datasets, S2, S1, joint latent, calibration
  evaluation/      # replay, fault injection, closed-loop suites
  configs/         # pinned model/runtime/robot bundles
```

Owner event-loop sketch

```
on_event(event):
    validate_schema_and_session(event)
    if event.is_stop:
        request_local_stop(); cancel_incompatible_work(); return
    if event.is_instruction_change:
        advance_instruction_revision(); invalidate_old_intent()
    if event.is_worker_result and not still_applicable(event):
        log_discard(event); return
    update_owned_record(event)
    if event.is_execution_ack:
        update_active_from_ack(event)
    publish_read_only_snapshot()
    dispatch_ready_work_without_waiting_for_inference()
```

Pseudocode, not an implemented safety controller. The local stop path must remain functional even if this event loop hangs. Input-schema errors do not authorize motion; malformed observations trigger the appropriate unavailable-evidence policy.

Worker scheduling

- S1 gets a bounded latest-observation input path. Do not block the control loop on logging or new model initialization.
- Use bounded fast/deep work queues, request-specific model caches, cancellation tokens, and expiring results. Background GPU kernels may not be preemptible; measure worst interference.
- Use immutable buffers or deliberate copy-on-write. A model returning early must not expose partially written latent tensors.
- Gate releases behind schema tests, deterministic simulator replays, numerical checks, and closed-loop evaluation. Log accepted and rejected changes so causality can be reconstructed.

Deployment and performance engineering

The report is evidence about one workload—not the whole control system.

The Fenon GR00T report describes A100 local warmed policy-call latency of 68.48 → 39.18 ms, with an offline numerical screen. It explicitly excludes network transport and robot control, and does not establish preserved closed-loop success. Treat its scheduling, image processing, decoding and precision changes as reusable engineering leads—not as Jev or physical-response measurements. [R12]

Placement	Recommended use / dependency
Local robot-side executor	Sensor acquisition, action timing, watchdog, physical stop interface. Must not depend on a cloud round trip for protective behavior.
Nearby GPU / workstation	S1 and potentially fast Jev; measure simultaneous demand. A separate process does not guarantee latency isolation.
H100 / training server	Training, profiling, teacher data and optional advisor. Do not infer available budget or production capacity.
Mac / simulation	Contracts, replay, simulation and supported lightweight models. Backend compatibility and memory must be measured.
T4 resources	Small-model/adapter experiments if supported. Two devices do not automatically become one contiguous memory pool.

Profiling order

Measure acquisition/resize/encode, model context work, readout, latent handoff, action generation, decoding, transport, queues and applied-command time. Use a synchronized complete-call benchmark as well as tracing. Test cold start separately from warmed steady state.

Model changes invalidate deployment assumptions

- Adding views, tokens, exits or a different latent shape may require engine re-export, profile changes, CUDA-graph recapture, and cache-key changes.
- Compare fixed and adaptive paths under real branching/batching. FLOP reductions and per-request early exits do not guarantee serving speedups.
- Quantization requires new output and calibration checks; small average action error or high cosine does not prove closed-loop noninferiority.

Keep the useful work Do not discard the existing GR00T optimization. Preserve it as a pinned executor baseline; then measure what changes when Jev, transport, and actual execution are added.

Evaluation: prove where Jev helps

All comparisons use matched tasks, instructions, sensors, and executor settings.

Comparison	Question it answers
Native VLA + full allowed intent/history vs Jev + same VLA	Does the supervisor add value beyond capabilities already present?
Finite-state / scripted supervisor vs Jev	Does learned selection help on ambiguity, new compositions and changed goals?
Oracle candidate set vs actual filter	How much failure comes from omitted valid operations?
Oracle state vs visual evidence	How much loss comes from perception rather than choice?
Sequential MJev→OneJev vs shared S2	Does removing the spatial-output handoff improve the accuracy/latency frontier?
Text vs learned latent vs corrupted latent	Does trained conditioning help; is S1 actually using it correctly?
One exit vs full depth vs adaptive depth	Does the gate save time without unacceptable decision risk?
No advisor vs useful / stale / misleading advisor	Has selective advice use been learned?
Canonical starts vs handoff-diverse training	Are executor transitions the main limit?

Report a compact, honest scorecard

Ordered task success; disturbance recovery; false completion; candidate recall; wrong-goal activation; stale-advice acceptance; useful-event-to-command p50/p95/p99; deadline misses; action discontinuity; compute per successful episode; risk versus abstention coverage; failure category.

Use paired environment seeds and independently held-out scenes/operators. Define success and noninferiority margins before examining the test set. Report counts and interval estimates; bootstrap by episode rather than treating correlated video frames as independent trials. Compare adaptive and fixed compute under matched resource budgets.

No assumed novelty Hierarchy, latent conditioning, typed choices, and adaptive depth already have precedents. The contribution must be the demonstrated method for grounded, timely revision under changing evidence—not a renamed block diagram.

Change-impact matrix

If X changes, these parts must change or be revalidated.

Implement X	Then change Y / Z	Required evidence
Jev latents instead of text	Train projection/receiver; align S2; version encoder–S1 bundle; revise export.	Intent adherence and task outcomes, including contradictory-latent tests.
Replace S1 or its action head	Regenerate action targets/transforms, normalization, chunks and guard mapping.	Atomic competence and transition success for that exact policy.
SO-100 to SO-101 or another arm	Verify kinematics, joint order/range, gripper scale, camera geometry and dynamics.	Calibration and embodiment-specific demonstrations/tests.
Add/remove/reposition cameras	Change preprocessing/masks, train visual input contract; recalibrate geometry.	View-dropout and small-target tests; no assumed stereo from two arbitrary views.
Add depth or 3D pose targets	Declare scale/frame/units; collect valid geometry labels; supply deployment sensor or estimator.	Metric error and downstream tolerance; no privileged sim depth leakage.
Early exits / recurrent loops	Train every readout and gate; stabilize latents per exit; adjust batching/engine.	Matched fixed-depth frontier and missed-refinement risk.
Skip crops or reduce visual tokens	Train the cheaper visual contract; preserve fine-detail tests.	Spatial and decision quality at the new resolution.
Concurrent advice / new advisor	Add source-time/context metadata; temporal and conflicting-guidance data.	Wrong/stale advice rejection; no direct execution authority.
Higher action rate / new horizon	Change dt, training resampling, prefix mask, controller constraints.	Actual reaction latency, smoothness and success—not sample frequency alone.
Quantize / compile / move to cloud	Rebuild engines, retest calibration and action drift, inject jitter/packet loss.	Closed-loop outcomes and end-to-end timing under concurrency.

All rows are implementation obligations proposed for this project. No change is guaranteed to improve performance. A failed branch can be retained as an ablation without becoming the deployed design.

Delivery plan and go/no-go gates

A complete program with evidence-based promotion—not a calendar estimate.

Work package	Deliverable	Promotion gate
WP0 — Freeze contracts	Pinned sources/models; action/camera schema; safety envelope; task definitions.	No unresolved unit, frame, embodiment or authority mismatch.
WP1 — Executor and replay	S1 baseline, simulator adapter, event log, protocol fault-injection harness.	Reliable isolated skills; stop/expiry/ACK behavior tested.
WP2 — Responsive typed supervision	Direct visual Jev; candidates; one state owner; text conditioning baseline.	Measured benefit or diagnosed limits versus native VLA and scripted supervisor.
WP3 — Temporal curriculum	Goal changes, handoffs, uncertainty, delayed/misleading advice; S1/S2 training.	Useful revisions and recovery improve on held-out episodes.
WP4 — Learned latent route	Selected-candidate z; receiver; joint/distilled alignment; interface tests.	Improves over text route without wrong-intent regression.
WP5 — Adaptive computation	One-exit/full-depth controls, trained gate; optional spatial worker.	Better measured quality–latency/compute frontier; no hidden S1 starvation.
WP6 — Integrated deployment	Pinned optimized runtime, network and concurrency tests, physical envelope trials.	Predeclared outcome and timing gates; guarded failure behavior verified.
WP7 — Learning from failures	Curated failure/recovery store; offline retraining and regression suite.	No unreviewed online weight changes; gains survive held-out and safety tests.

Remaining assumptions to resolve through inspection or measurement

Exact available arm/cameras; action mode and controller dt; available training budget; supported runtime backends; candidate extraction; per-skill coverage; actual latency distribution. These do not prevent defining the full architecture, but they block claims of deployment readiness.

Release artifact bundle

Compatible model + projection + normalization + schema revisions; reproducible configs; dataset manifests and split hashes; tests and experiment logs; supported capability registry; operating envelope; rollback checkpoint; unresolved-risk register.

Final decision Keep whichever route wins the tests. If the native VLA or a simple supervisor satisfies the requirements at lower cost, Jev should earn a narrower role or be removed from that path. The objective is useful, revisable behavior—not preserving a favorite architecture.

Sources and evidence boundaries

Primary references. Links are clickable in the Word document and PDF.

[R01] Figure — Helix: A Vision-Language-Action Model for Generalist Humanoid Control

https://www.figure.ai/news/helix

Published 20 February 2025. Evidence: dual-rate observation-conditioned S1/S2, latent interface, joint loss, temporal offset. Company-reported architecture, not independently reproduced here.

[R02] Figure — Helix Accelerating Real-World Logistics

https://www.figure.ai/news/helix-logistics

Published 26 February 2025. Evidence: stereo/multiscale S1 design and corrective-data curation. Do not transfer its rates, throughput or speed-resampling results to a different arm.

[R03] OmniJev — OneJev repository / README

https://github.com/OmniJev/OneJev/blob/main/README.md

Typed multimodal interfaces, model/training implementation. Pin a concrete revision before use; benchmark calibration is not a robot-success guarantee.

[R04] MJev-Spatial: System One Spatial Intelligence via Native Attention Matching

https://arxiv.org/html/2610.22276v1

Architecture and refinement procedures. Shared scoring does not make all output types single-stage or provide metric-geometry guarantees.

[R05] LEV — Thinking Fast and Deep with Looped Decision Models

https://openlair.github.io/lev/

Author project page; arXiv:2610.20951. Natively looped backbones and depth readouts; deepest-answer agreement is not correctness. No ready-made multimodal robotic adaptation established.

[R06] KAME and randomized intermediate guidance

https://arxiv.org/html/2510.02327v1

KAME: learned asynchronous speech guidance. Follow-up: https://arxiv.org/abs/2609.30773. Selective advice training is a transfer hypothesis for robotic supervision, not a demonstrated robotic result.

[R07] Physical Intelligence — openpi

https://github.com/Physical-Intelligence/openpi

Open model implementations, pretrained/fine-tuned policies, data transforms and deployment paths. A new embodiment or conditioning interface requires adaptation and validation.

Evidence discipline Reported architectures are distinguished from proposed adapters and training recipes. No imported benchmark establishes a breakthrough, a hard real-time guarantee, or physical safety for this system.

Sources, caveats, and reproducibility

Complete the experiment—not just the diagram.

[R08] Hugging Face — SmolVLA

https://huggingface.co/blog/smolvla

Visuomotor policy and efficient action-model reference. A compact architecture does not guarantee the project's end-to-end timing targets.

[R09] LeRobot — pi052_robocasa model card

https://huggingface.co/lerobot/pi052_robocasa

Checkpoint-specific camera/state/action interface. Do not equate its RoboCasa contract with an SO arm or infer validated skill count from strings.

[R10] Physical Intelligence — Real-Time Action Chunking

https://www.pi.website/research/real_time_chunking

Committed-prefix and asynchronous-chunk compatibility. Training-time follow-up: https://arxiv.org/abs/2512.05964. Arbitrary instruction changes need additional transition validation.

[R11] NVIDIA — Isaac GR00T N1 architecture

https://developer.nvidia.com/blog/accelerate-generalist-humanoid-robot-development-with-nvidia-isaac-gr00t-n1/

Open VLM/action-system architecture reference. N1 design descriptions are not a specification for every later checkpoint or a plug-compatible Jev interface.

[R12] Fenon — GR00T N1.7 × A100 + H100 report

https://fenon-gr00t-a100-report.vercel.app/

Report reviewed; described experiments dated 18–19 October 2026. Local offline results only. Referenced Fenon repository was not audited in this plan; no code-level validation is claimed.

[R13] Thinking Machines Lab — Interaction Models

https://thinkingmachines.ai/blog/interaction-models/

Evidence: responsive frontend and background computation. Not used here to assert checkpoint equivalence, exact robot timing, or an implementation recipe.

[R14] RoboCombine — Exploring the Limits of Combinatorial Generalization in Embodied Manipulation

https://arxiv.org/abs/2610.15425

Prior source discussed in the conversation; useful additional reading on sequencing/handoff analysis. The rollout and transition requirements in this plan are proposed evaluation obligations.

[R15] CALM — Confident Adaptive Language Modeling

https://arxiv.org/abs/2207.07061

Additional early-exit design reference. Its language-model result is not evidence that an arbitrary multimodal layer can safely replace the full model.

Document status: research implementation specification, 20 October 2026. Numerical tensor examples and curriculum choices are proposals. The figures are original conceptual diagrams, not vendor architecture reproductions. Every promoted configuration requires a pinned implementation, causal training data, and matched evaluation.